# Mardi 13/10 — Premiers pas en JavaScript avec Node

Kevin Bardoux

mardi 13 octobre 2026

## Table des matières

Objectifs ..... 2
Matin — Premiers pas en JavaScript avec Node : installation, console, variables, types, === ..... 2
Rappel (5 min) ..... 2
Apport 1 — Installer Node.js (10 min) ..... 3
Node.js — attention et vérification ..... 5
Apport 2 — Un éditeur de code : VS Code (10 min) ..... 5
VS Code — attention et vérification ..... 8
Apport 3 — `const` et `let` (8 min) ..... 8
Variables — attention et vérification ..... 10
Apport 4 — Les types et `typeof` (7 min) ..... 10
Types — attention et vérification ..... 11
Exercice — Prédire, puis vérifier (30 min) ..... 11
Apport 5 — Comparer avec `===` (10 min) ..... 12
`===` — attention et vérification ..... 13
Exercice — Quatre comparaisons (10 min) ..... 13
Si le temps le permet — Quatre pièges de `==` ..... 13
À retenir — ce matin ..... 14
Débrief ..... 14
Après-midi — TP — « Hello, JS » · premiers tests automatiques . 14
Récupérer les fichiers (10 min) ..... 14
Apport 6 — Lancer un programme et ses tests (15 min) ..... 15
Tests — attention et vérification ..... 18
Apport 7 — Une valeur dans un texte (5 min) ..... 18

Littéral de gabarit — attention et vérification ..... 19
Exercices 01 à 04 (1 h 15) ..... 19
Critères de réussite ..... 20
Si le temps le permet — Exercice 05 : prix TTC ..... 20
Pour aller plus loin ..... 20
À retenir — cet après-midi ..... 21
Débrief (10 min) ..... 21
Sources ..... 21

> Note
>
> **Compétences visées** : installer son environnement · résolution de problème · apprendre en continu **Livrable** : exercices 01 à 04 au vert, une capture du terminal.

## Objectifs

- Installer Node.js et VS Code, et lancer un programme dans le terminal.
- Utiliser `const`, `let`, `typeof` et `console.log`.
- Comparer avec `===` et expliquer pourquoi on évite `==`.
- Lancer des tests automatiques et lire un test rouge.

**Comment se passe la journée.** Chaque notion suit le même ordre : pourquoi, définition, exemple, pas à pas, erreur fréquente, vérification. Un encadré « À retenir » termine chaque demi-journée. Le formateur montre d'abord ; vous faites ensuite le premier exercice tous ensemble ; puis vous continuez seuls. ESLint, prévu au programme de la semaine, arrive vendredi matin.

# Matin — Premiers pas en JavaScript avec Node : installation, console, variables, types, ===

### Rappel (5 min)

1. À quoi sert une variable ?
2. Que faut-il pour qu'un message démarre un script ?

3. Qu’est-ce que le pseudo-code ?

Trois questions courtes, sans regarder vos notes : la veille, la semaine passée, puis plus loin. Répondez sur une feuille ; la correction suit, à l’oral.

## Apport 1 — Installer Node.js (10 min)

- **Pourquoi** : lancer vos programmes JavaScript sur votre ordinateur.
- **Node.js** exécute du JavaScript en dehors du navigateur.

**Pourquoi.** Cette semaine, vous écrivez des programmes et vous les faites vérifier par des tests, sur votre ordinateur. Le navigateur lit le JavaScript d’une page web, mais il ne lance pas un programme seul dans un terminal.

**Définition.** Node.js est un programme qui exécute du JavaScript en dehors du navigateur, dans un terminal.

On installe la **version LTS** (*Long Term Support*) : elle est maintenue longtemps, et c’est celle que Node.js recommande pour un usage sérieux (OpenJS Foundation, s. d.-b). La page de téléchargement officielle indique laquelle est active, et propose l’installeur qui correspond à votre système (OpenJS Foundation, s. d.-a).

**Exemple.** Une fois Node.js installé, ces deux commandes affichent chacune un numéro de version (pour Node, la lettre `v` suivie de chiffres).

Les blocs à fond sombre reproduisent l’écran du terminal de Windows (PowerShell). La ligne qui commence par `PS C:\Users\stagiaire>` est l’**invite** : le dossier où le terminal travaille, suivi de la commande tapée. Ne tapez que ce qui suit `>`. Les lignes suivantes sont la réponse du terminal. Sous macOS et Linux, l’invite est différente, la réponse est la même.

```
PS C:\Users\stagiaire> node -v
v24.21.0
PS C:\Users\stagiaire> npm -v
11.19.0
```

Vos numéros peuvent être un peu différents : l'installation est réussie si chaque commande affiche un numéro, et si celui de Node commence par `v24` ou plus.

**Pas à pas.** Le formateur fait chaque étape au vidéoprojecteur ; vous la refaites aussitôt sur votre ordinateur, puis vous attendez l'étape suivante (35 min).

## Windows

1. Ouvrez nodejs.org, page *Download* ; gardez la version **LTS**.
2. Téléchargez l'installeur **Windows Installer (.msi)**.
3. Lancez-le. Acceptez l'autorisation de modifier l'ordinateur : vous êtes administrateur de votre poste.
4. Gardez les choix proposés, mais laissez **décochée** la case « Tools for Native Modules ».
5. Terminez l'installation.
6. Fermez PowerShell s'il était ouvert, puis rouvrez-le.
7. Tapez `node -v`, puis Entrée ; puis `npm -v`, puis Entrée.

## macOS

1. Ouvrez nodejs.org, page *Download* ; gardez la version **LTS**.
2. Téléchargez l'installeur **macOS Installer (.pkg)**.
3. Lancez-le et suivez les étapes, avec les choix proposés.
4. Fermez le Terminal s'il était ouvert, puis rouvrez-le.
5. Tapez `node -v`, puis Entrée ; puis `npm -v`, puis Entrée.

## Linux

1. Ouvrez un terminal.
2. Installez fnm : `curl -fsSL https://fnm.vercel.app/install | bash`
3. Fermez le terminal, puis rouvrez-le.
4. Installez la LTS : `fnm install --lts`
5. Tapez `node -v`, puis Entrée ; puis `npm -v`, puis Entrée.

L'installeur installe aussi **npm**, le gestionnaire de paquets de Node, qui servira vendredi. La case « Tools for Native Modules » (outils pour les modules natifs) installe Chocolatey et des outils de compilation, inutiles pour la formation, et rallonge beaucoup

l'installation. Sous Linux, **fnm** est un gestionnaire de versions de Node : il installe la LTS du moment et permet d'en changer en une commande (Schniz, s. d.-b, s. d.-a). Le paquet `nodejs` d'une distribution est souvent en retard de plusieurs versions (OpenJS Foundation, s. d.-b) ; une alternative courante à fnm est nvm (nvm-sh, s. d.).

**En secours : StackBlitz.** Si l'installation échoue, StackBlitz fait tourner Node.js dans le navigateur grâce aux WebContainers (StackBlitz, s. d.-a) ; utilisez Chrome ou un navigateur basé sur Chromium (Firefox et Safari sont seulement en bêta) (StackBlitz, s. d.-b). Déposez-y les fichiers de l'après-midi et ouvrez son terminal intégré. L'installation se termine mercredi.

## Node.js — attention et vérification

> ⚠ Attention
>
> « n'est pas reconnu » ou « commande introuvable » : le terminal était ouvert avant l'installation. Fermez-le, rouvrez-le, retapez `node -v`.

- **Vérifiez** : `node -v` et `npm -v` affichent chacun un numéro.

## Apport 2 — Un éditeur de code : VS Code (10 min)

- **Pourquoi** : écrire vos fichiers `.js` et les retrouver.
- **VS Code** : l'éditeur de code de la formation, gratuit.

**Pourquoi.** Cet après-midi, vous modifiez des fichiers `.js`. Un traitement de texte ajoute une mise en forme que Node ne sait pas lire, et le Bloc-notes peut enregistrer `hello.js` sous le nom `hello.js.txt`. Un éditeur de code enregistre du texte brut, colore le code et numérote les lignes.

**Définition.** VS Code (Visual Studio Code, de Microsoft) est un éditeur de code : un logiciel gratuit pour écrire et enregistrer des fichiers de programmation. C'est l'éditeur de toute la formation.

**Exemple.** Un fichier `essai.js`, créé dans VS Code et enregistré dans `dwwm/semaine-02`, qui contient une ligne :

```
console.log('Bonjour depuis VS Code');
```

**Pas à pas.** Le formateur fait chaque étape ; vous la refaites aussitôt. Jusqu'en semaine 4, VS Code s'affiche en anglais : les noms des menus sont donc en anglais.

1. **Installer VS Code.**

## Windows

1. Ouvrez la page de téléchargement de VS Code (Microsoft, s. d.-b) ; cliquez sur **Windows**.
2. Lancez le fichier téléchargé ; acceptez le contrat de licence.
3. Gardez les choix proposés ; terminez l'installation (Microsoft, s. d.-e).

## macOS

1. Ouvrez la page de téléchargement de VS Code (Microsoft, s. d.-b) ; cliquez sur **Mac**.
2. Ouvrez le fichier téléchargé.
3. Glissez **Visual Studio Code.app** dans le dossier **Applications** (Microsoft, s. d.-d).

## Linux

1. Ouvrez la page de téléchargement de VS Code (Microsoft, s. d.-b) ; téléchargez le paquet **.deb** (Ubuntu, Debian).
2. Dans le terminal, depuis le dossier du fichier : `sudo apt install ./<nom-du-fichier>.deb` (Microsoft, s. d.-c).

2. **Désactiver les fonctions IA (palier 0).** Ouvrez les réglages avec `Cmd+,` (mac), `Ctrl+,` (windows). Dans la barre de recherche, tapez `chat.disableAIFeatures`. Cochez la case. Ne changez aucun autre réglage (Microsoft, s. d.-f).
3. **Afficher les extensions des noms de fichiers.**

## Windows

- Windows 11 : dans l'Explorateur de fichiers, menu **Affichage**, puis **Afficher**, puis **Extensions de noms de fichiers**.

- Windows 10 : dans l'Explorateur de fichiers, onglet **Affichage**, groupe **Afficher/Masquer**, cochez **Extensions de noms de fichiers** (Microsoft, s. d.-a).

### macOS

Dans le Finder, menu **Finder**, puis **Réglages**, puis **Avancés** ; cochez **Afficher toutes les extensions de fichiers** (Apple, s. d.).

### Linux

Les gestionnaires de fichiers de Linux affichent déjà les extensions : rien à faire.

4. **Créer un fichier.** Dans VS Code : *File*, puis *New File...* ; tapez `essai.js`, puis Entrée.
5. **Enregistrer.** Choisissez le dossier `dwwm/semaine-02` (créez `semaine-02` s'il n'existe pas), puis *Save*.
6. **Écrire.** Tapez la ligne de l'exemple, puis enregistrez avec `Cmd+S` (mac), `Ctrl+S` (windows).
7. **Fermer et rouvrir.** Fermez VS Code. Rouvrez-le, puis *File*, *Open Folder...*, et choisissez le dossier `dwwm/semaine-02`.
8. **Faire confiance au dossier.** VS Code demande : « Do you trust the authors of the files in this folder? » (« Faites-vous confiance aux auteurs des fichiers de ce dossier ? »). Cliquez sur **Yes, I trust the authors** (« Oui, je fais confiance aux auteurs »).

   **Vous devez voir** : la fenêtre de la question se ferme, et les fichiers du dossier, dont `essai.js`, apparaissent à gauche.

> ⚠ Attention
>
> Ne faites confiance qu'aux dossiers du cours ou à ceux que vous avez créés vous-même. En cas de doute, cliquez sur **No, I don't trust the authors** (« Non, je ne fais pas confiance aux auteurs ») : VS Code ouvre le dossier en mode restreint, qui bloque l'exécution automatique de code (Microsoft, s. d.-g). Puis demandez au formateur.

9. **Lancer.** Dans le terminal : `cd ~/dwwm/semaine-02`, puis `node essai.js`. Le terminal affiche `Bonjour depuis VS Code`.

## VS Code — attention et vérification

> ⚠️ Attention
>
> `Cannot find module` alors que le fichier existe : il s'appelle peut-être `essai.js.txt`. Affichez les extensions, puis renommez-le `essai.js`.

- **Vérifiez** : `node essai.js` affiche `Bonjour depuis VS Code`.

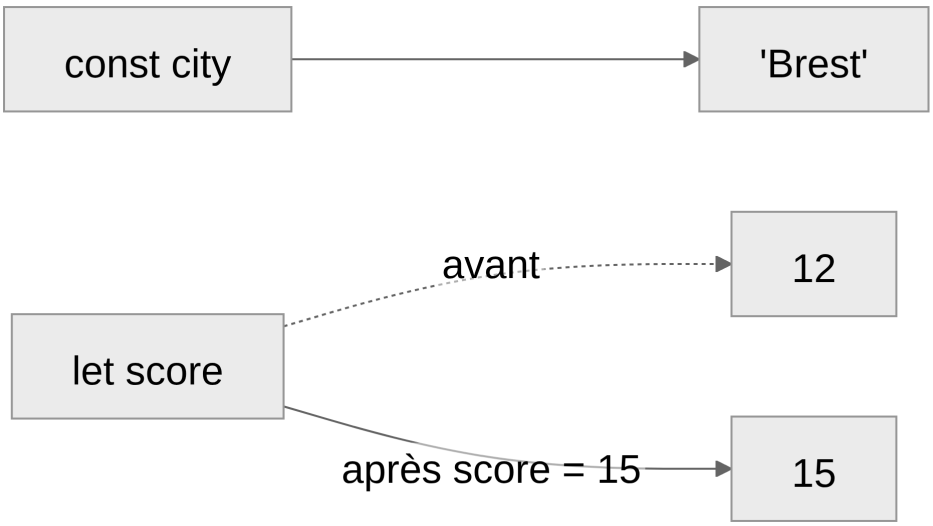
## Apport 3 — `const` et `let` (8 min)

- **Pourquoi** : garder une valeur pour la réutiliser plus loin.
- **Variable** : un nom qui désigne une valeur ; `const` par défaut.

**Pourquoi.** Un programme calcule une valeur, puis s'en sert plus loin : un prénom, un âge, un score. Pour la retrouver, on lui donne un nom.

**Définition.** Une variable est un nom qui désigne une valeur. On la déclare avec `const` (le nom désigne toujours la même valeur) ou avec `let` (le nom peut ensuite désigner une autre valeur) (MDN Web Docs, s. d.-e). Règle du cours : `const` par défaut, `let` seulement si la valeur doit changer.

**Une image, avec sa limite.** Pour s'en souvenir, on peut imaginer une **étiquette** portant le nom, collée sur une valeur : `const` colle l'étiquette pour de bon, `let` permet de la décoller et de la recoller ailleurs. Limite de l'image : en semaine 6, deux étiquettes pourront désigner le même objet, et modifier l'objet par l'une se verra par l'autre ; l'image sera alors complétée.




Une image : la variable est une étiquette collée sur une valeur. const ne se recolle pas, let peut se recoller.

**Exemple.**

```
const city = 'Brest';
let score = 12;
score = 15;
console.log(city, score); // Brest 15
```

**Pas à pas.** Le formateur tape chaque ligne au vidéoprojecteur ; vous la retapez aussitôt dans votre terminal.

1. Tapez `node`, puis Entrée. L'invite `>` apparaît : vous êtes dans le **REPL**, la console interactive de Node.
2. Tapez `const city = 'Brest'`, puis Entrée. Le REPL affiche `undefined` : c'est normal, une déclaration ne renvoie aucune valeur.
3. Tapez `city`, puis Entrée. Le REPL affiche `'Brest'`.
4. Tapez `let score = 12`, puis Entrée ; puis `score = 15`, puis Entrée. Le REPL affiche `15`.
5. Tapez `console.log(city, score)`, puis Entrée. Le REPL affiche `Brest 15`, puis `undefined` (la valeur renvoyée par `console.log`, à ignorer) (MDN Web Docs, s. d.-a).
6. Tapez `.exit`, puis Entrée, pour quitter le REPL.

## Variables — attention et vérification

> ⚠ Attention
>
> `TypeError: Assignment to constant variable.` : vous avez donné une nouvelle valeur à une `const`. Déclarez-la avec `let`, seulement si sa valeur doit changer.

- **Vérifiez** : `let n = 10; n = n + 5;` : que vaut `n` ?
- Prédisez, puis tapez la ligne dans le REPL.

## Apport 4 — Les types et `typeof` (7 min)

- **Pourquoi** : `'4' + 2` ne donne pas `6`.
- **Type** : la sorte d'une valeur ; `typeof` la donne.

**Pourquoi.** Le même signe ne fait pas la même chose selon les valeurs : `4 + 2` donne `6`, mais `'4' + 2` donne `'42'`. Pour comprendre un résultat, il faut connaître le type de chaque valeur.

**Définition.** Le type d'une valeur dit de quelle sorte elle est : un nombre (`number`), un texte (`string`), vrai ou faux (`boolean`)… L'opérateur `typeof` renvoie ce type, sous forme de texte (MDN Web Docs, s. d.-c). Un texte s'écrit entre guillemets : `'42'` est un texte, `42` est un nombre.

**Exemple.**

```
typeof 42      // 'number'
typeof '42'    // 'string'
typeof true    // 'boolean'
'4' + 2        // '42' : + colle deux textes
'4' * 2        // 8 : * convertit le texte en nombre
```

**Pas à pas.** Le formateur tape chaque ligne au vidéoprojecteur ; vous la retapez aussitôt.

1. Ouvrez le REPL : `node`, puis Entrée.
2. Tapez `typeof 42`, puis Entrée. Le REPL affiche `'number'`.
3. Tapez `typeof '42'`, puis Entrée. Le REPL affiche `'string'` : les guillemets font un texte.
4. Tapez `'4' + 2`, puis Entrée. Le REPL affiche `'42'`, entre guillemets : c'est un texte.

5. Tapez `'4' * 2`, puis Entrée. Le REPL affiche `8`, sans guillemets : c'est un nombre.
6. Tapez `.exit`, puis Entrée.

Le texte placé après `//` est un **commentaire** : Node l'ignore. JavaScript accepte les guillemets simples (`'…'`) et doubles (`"…"`) ; le cours utilise les simples dans le code.

## Types — attention et vérification

> ⚠ Attention
>
> `+` avec un texte colle au lieu d'additionner : `'4' + 2` donne `'42'`. Vous le voyez aux guillemets du résultat. Convertissez d'abord : `Number('4') + 2` donne `6`.

- **Vérifiez** : que donne `typeof 'true'` ?
- Prédisez, puis tapez-le dans le REPL.

## Exercice — Prédire, puis vérifier (30 min)

1. Ensemble (5 min) : prédisez à voix haute les lignes 1 et 7.
2. Seuls, sur papier (10 min) : prédisez les lignes 2 à 9.
3. Dans le REPL (10 min) : tapez chaque ligne, comparez.
4. Lignes 7 à 9 en étiquettes, sur papier (5 min).
5. **Réussi si** : chaque erreur de prédiction a sa raison écrite.
6. `typeof 42`
7. `typeof '42'`
8. `typeof true`
9. `typeof undefined`
10. `'4' + 2`
11. `'4' * 2`
12. `let x = 5; x = x + 1; x`
13. `const y = 3; y = 4`
14. `let a = 5; let b = a; a = 7; b`

**Les étiquettes sur papier** : dessinez la mémoire pendant que le programme avance, comme le schéma de l'apport sur les variables.

1. Recopiez la ligne 7 en mettant une instruction par ligne.
2. Sous chaque instruction, dessinez les étiquettes : un nom, une flèche, une valeur.
3. Quand une étiquette est recollée, barrez l'ancienne flèche et tracez la nouvelle.
4. Recommencez avec les lignes 8, puis 9. À la ligne 9, écrivez la valeur de `b` après `a = 7`.
5. Vérifiez la valeur finale de chaque ligne dans le REPL.

**Réussi si** : la valeur entourée sur votre dessin est celle qu'affiche le REPL.

## Apport 5 — Comparer avec `===` (10 min)

- **Pourquoi** : un programme doit savoir si deux valeurs sont égales.
- **`===`** : même type **et** même valeur ; jamais `==`.

**Pourquoi.** Un programme compare sans cesse : la réponse est-elle la bonne ? L'âge vaut-il 18 ? Un test vérifie que le résultat obtenu est égal au résultat attendu.

**Définition.** `===` (égalité stricte) renvoie `true` si les deux valeurs ont le même type et la même valeur, sinon `false`. `==` tente d'abord une conversion vers un type commun avant de comparer (MDN Web Docs, s. d.-b) : c'est la source des surprises. Règle du cours : toujours `===`, et `!==` pour « différent ».

**Exemple.**

```
3 === 3     // true
'3' === 3   // false : un texte, un nombre
'3' == 3    // true : == a converti le texte
3 !== 4     // true : 3 est différent de 4
```

**Pas à pas.** Le formateur tape chaque ligne ; vous la retapez aussitôt.

1. Ouvrez le REPL : `node`, puis Entrée.
2. Tapez `3 === 3`, puis Entrée : `true`.
3. Tapez `'3' === 3`, puis Entrée : `false`, car un texte n'est jamais strictement égal à un nombre.
4. Tapez `'3' == 3`, puis Entrée : `true`, car `==` a converti le texte en nombre avant de comparer.
5. Tapez `3 !== 4`, puis Entrée : `true`.
6. Tapez `.exit`, puis Entrée.

## `===` — attention et vérification

> ⚠️ Attention
>
> Un seul `=` ne compare pas : il donne une valeur. `score = 3` change `score` ; `score === 3` pose la question. Une variable qui change sans raison : cherchez un `=` seul.

- **Vérifiez** : que donne `'5' === 5` ?
- Prédisez, puis tapez-le dans le REPL.

## Exercice — Quatre comparaisons (10 min)

1. Ensemble (2 min) : prédisez à voix haute la comparaison 1.
2. Seuls (5 min) : écrivez `true` ou `false` pour 2 à 4.
3. Dans le REPL (3 min) : vérifiez chaque comparaison.
4. **Réussi si** : chaque surprise est entourée et expliquée.
5. `3 === 3`
6. `'3' === 3`
7. `'3' == 3`
8. `0 == ''`

### Si le temps le permet — Quatre pièges de `==`

5. `0 === ''`
6. `null == undefined`
7. `null === undefined`
8. `true == 1`

## À retenir — ce matin

> À retenir
>
> - **Node.js** lance du JavaScript dans un terminal.
> - **VS Code** : un éditeur, des fichiers `.js` sans `.txt`.
> - **Variables** : `const` par défaut, `let` si la valeur change.
> - **Types** : `+` avec un texte colle ; `Number(…)` convertit.
> - **Comparer** : `===` compare ; `=` donne une valeur.

Le détail, notion par notion :

- **Node.js** : il lance du JavaScript dans un terminal ; on installe la version LTS ; `node -v` vérifie l'installation.
- **VS Code** : c'est l'éditeur de la formation ; les fonctions IA sont désactivées (`chat.disableAIFeatures`) ; affichez les extensions pour repérer un `.js.txt`.
- **Variables** : une variable est un nom qui désigne une valeur ; `const` par défaut, `let` si la valeur doit changer ; une `const` ne reçoit jamais de nouvelle valeur.
- **Types** : chaque valeur a un type, que `typeof` donne ; `'42'` est un texte, `42` un nombre ; `+` avec un texte colle, `Number(…)` convertit.
- **Comparer** : `===` compare le type et la valeur ; `=` donne une valeur, `===` compare ; jamais `==` dans le cours.

### Débrief

- Quelle différence entre `let` et `const` ?
- Pourquoi `'4' + 2` donne-t-il `'42'` ?
- Pourquoi préférer `===` ?

# Après-midi — TP — « Hello, JS » · premiers tests automatiques

### Récupérer les fichiers (10 min)

1. Téléchargez l'archive des exercices.
2. Extrayez-la : un dossier `s02-exercices` apparaît.
3. Déplacez les fichiers de ce dossier dans `dwwm/semaine-02` (le dossier de `essai.js`).

4. Dans VS Code : *File*, *Open Folder*..., puis `dwwm/semaine-02`.
5. Dans le terminal : `cd ~/dwwm/semaine-02`, puis `ls`.
6. **Réussi si** : `ls` affiche `01-hello.js` et les autres fichiers.

Pour extraire l'archive (étape 2) : sous Windows, clic droit sur le fichier, *Extraire tout*..., puis *Extraire* ; sous macOS, double-cliquez sur le fichier ; sous Linux, clic droit, *Extraire ici*. Le dossier `dwwm` est dans votre dossier personnel.

La commande `cd ~/dwwm/semaine-02` s'écrit de la même façon dans PowerShell, sous macOS et sous Linux : `~` désigne votre dossier personnel. `cd` veut dire *change directory* : le terminal travaille désormais dans ce dossier. `ls` affiche son contenu.

## Apport 6 — Lancer un programme et ses tests (15 min)

- **Pourquoi** : vérifier un programme sans tout relire à la main.
- **Test** : il lance votre programme et compare au résultat attendu.

**Pourquoi.** Relire un programme ne suffit pas à savoir s'il fait ce qui est demandé. Un test le vérifie pour vous, de la même façon à chaque fois, en une commande.

**Définition.** Un test est un petit programme qui lance le vôtre et compare ce qu'il affiche (le résultat obtenu) avec ce qu'il devrait afficher (le résultat attendu). Chaque exercice a son fichier de tests, qui finit par `.test.js` (OpenJS Foundation, s. d.-d). `node 01-hello.js` lance **un** programme ; `node --test 01-hello.test.js` lance **ses** tests.

**Exemple.** Un test rouge, au départ de l'exercice 01 : `+ actual` est ce que votre programme affiche, `- expected` ce qu'il **devrait** afficher. Voici tout ce que le terminal affiche ; les lignes utiles sont ✖ `affiche Hello, world!`, puis les lignes `+` et `-`. Les lignes qui commencent par `at` et la suite entre accolades sont des détails pour plus tard : ignorez-les.

```
PS C:\Users\stagiaire\dwwm\semaine-02> node --test 01-
hello.test.js
```

```
✖ affiche Hello, world! (133.07442ms)
ℹ tests 1
ℹ suites 0
ℹ pass 0
ℹ fail 1
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 376.949659

✖ failing tests:

test at 01-hello.test.js:13:1
✖ affiche Hello, world! (133.07442ms)
  AssertionError [ERR_ASSERTION]: Expected values to be
strictly equal:
  + actual - expected

  + 'À faire'
  - 'Hello, world!'

      at TestContext.<anonymous> (file:///C:/Users/
stagiaire/dwwm/semaine-02/01-hello.test.js:14:10)
      at Test.runInAsyncScope (node:async_hooks:227:14)
      at Test.run (node:internal/test_runner/
test:1402:25)
      at Test.start (node:internal/test_runner/
test:1262:17)
      at startSubtestAfterBootstrap (node:internal/
test_runner/harness:387:17) {
    generatedMessage: true,
    code: 'ERR_ASSERTION',
    actual: 'À faire',
    expected: 'Hello, world!',
    operator: 'strictEqual',
```

```
    diff: 'simple'
  }
```

**Pas à pas.** Le formateur fait chaque étape au vidéoprojecteur sur `01-hello.js` ; vous la refaites aussitôt.

1. Tapez `node 01-hello.js`, puis Entrée. Le programme affiche `À faire` : c'est le fichier de départ.
2. Tapez `node --test 01-hello.test.js`, puis Entrée. Une croix rouge ✖ apparaît : le test échoue. C'est normal au départ.
3. Cherchez les lignes qui commencent par `+` et par `-`. `+` : ce que votre programme affiche (`'À faire'`). `-` : ce qu'il devrait afficher (`'Hello, world!'`).
4. Dans VS Code, ouvrez `01-hello.js`. Lisez l'énoncé, en commentaire en haut du fichier.
5. Remplacez `'À faire'` par `'Hello, world!'`. Enregistrez avec `Cmd+S` (mac), `Ctrl+S` (windows).
6. Relancez `node --test 01-hello.test.js`. Une coche verte `✓` apparaît : le test passe.

**Vous devez voir** : une coche `✓` devant le nom du test, et `ℹ fail 0`.

```
PS C:\Users\stagiaire\dwwm\semaine-02> node --test 01-
hello.test.js
✓ affiche Hello, world! (109.631557ms)
ℹ tests 1
ℹ suites 0
ℹ pass 1
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 357.329955
```

Les durées entre parenthèses changent à chaque lancement : elles ne comptent pas.

La fiche Lire un message d'erreur montre ce test rouge et les autres messages de Node, partie par partie.

`node --test` sans nom de fichier lance **tous** les tests du dossier, y compris ceux des exercices de vendredi : beaucoup de croix rouges, c'est normal. Pour un seul exercice, donnez le nom de son fichier de tests.

**Les lignes fournies : `process.argv`.** `node 02-presentation.js Ada 36` lance le programme avec deux mots en plus. `process.argv` contient les arguments de la ligne de commande : le chemin de Node, celui du fichier, puis ce que vous tapez ensuite (OpenJS Foundation, s. d.-c). Ici, `process.argv[2]` vaut `'Ada'` et `process.argv[3]` vaut `'36'` : ce sont des **textes**, que `Number(…)` convertit. Ces lignes sont fournies en haut des exercices : ne les modifiez pas. Vous écrirez vos propres fonctions la semaine prochaine.

## Tests — attention et vérification

> ⚠ Attention
>
> `Could not find` ou `tests 0` : le terminal n'est pas dans le bon dossier. Tapez `cd ~/dwwm/semaine-02`, puis relancez.

- **Vérifiez** : que montre la ligne qui commence par `-` ?
- Répondez sur votre feuille, puis comparez avec votre voisin.

## Apport 7 — Une valeur dans un texte (5 min)

- **Pourquoi** : afficher une phrase qui contient une valeur.
- **Littéral de gabarit** : un texte entre accents graves ; `${…}` insère.

**Pourquoi.** L'exercice 02 doit afficher une phrase qui contient un prénom et un âge. Coller des morceaux avec `+` est long et source d'erreurs.

**Définition.** Un littéral de gabarit est un texte écrit entre accents graves (`` ` ``), dans lequel `${…}` insère la valeur d'une variable ou d'un calcul (MDN Web Docs, s. d.-d).

**Exemple.**

```
const firstName = 'Ada';
console.log(`Bonjour ${firstName} !`); // Bonjour Ada !
```

**Pas à pas.** Le formateur tape chaque ligne dans le REPL ; vous la retapez aussitôt.

1. Ouvrez le REPL : `node`, puis Entrée.
2. Tapez `const firstName = 'Ada'`, puis Entrée.
3. Tapez `console.log(`Bonjour ${firstName} !`)`, puis Entrée. Le REPL affiche `Bonjour Ada !`.
4. Tapez `console.log(`L'an prochain : ${36 + 1} ans`)`, puis Entrée. Le REPL affiche `L'an prochain : 37 ans` : on peut aussi calculer dans `${…}`.
5. Tapez `.exit`, puis Entrée.

L'accent grave s'obtient sous Windows (clavier AZERTY) avec `AltGr+7` (windows), puis la touche Espace ; sous macOS, avec la touche située à gauche de Entrée (clavier AZERTY).

## Littéral de gabarit — attention et vérification

> ⚠ Attention
>
> Avec des guillemets `'…'`, `${firstName}` s'affiche tel quel. Seuls les accents graves `` `…` `` remplacent `${…}` par la valeur.

- **Vérifiez** : que donne `` `${2 + 3} ans` `` ?
- Prédisez, puis tapez-le dans le REPL.

## Exercices 01 à 04 (1 h 15)

1. Ensemble (10 min) : `01-hello`, du test rouge au test vert.
2. Seuls : `02-presentation`, puis `03-types`, puis `04-egalite`.
3. Pour chacun : lisez l'énoncé, lancez, puis lancez ses tests.
4. **Réussi si** : ✓ pour les tests de 01 à 04.

| Exercice | Niveau | Ce qu'il fait travailler |
|---|---|---|
| `01-hello` | facile | afficher `Hello, world!` |
| `02-presentation` | facile | un littéral de gabarit `` `…${…}…` `` |
| `03-types` | facile | `typeof`, et `+` sur du texte ou des nombres |
| `04-egalite` | moyen | calculer des comparaisons avec `===` |

Pour chaque exercice, dans cet ordre :

1. Ouvrez le fichier dans VS Code (par exemple `02-presentation.js`) et lisez l'énoncé, en commentaire en haut.
2. Écrivez votre code à la place de la ligne indiquée par `TODO`. Enregistrez.
3. Lancez le programme : `node 02-presentation.js Ada 36`.
4. Lancez ses tests : `node --test 02-presentation.test.js`.
5. Tant qu'une croix rouge reste, lisez les lignes `+` et `-`, corrigez, enregistrez, relancez.

> **Indice — 02-presentation**
>
> Calculez d'abord l'âge de l'an prochain dans une constante, puis affichez la phrase avec un littéral de gabarit. Si vous obtenez `361` au lieu de `37`, c'est que vous additionnez du texte : vérifiez que vous utilisez bien la constante `age` (déjà convertie en nombre).

> **Indice — 04-egalite**
>
> Un littéral de gabarit peut contenir une comparaison : `${a === b}` affiche `true` ou `false`.

**Critères de réussite**

- ☐ `node -v` affiche une version (ou le plan B fonctionne)
- ☐ `essai.js` s'ouvre dans VS Code et s'exécute
- ☐ Les tests de 01 à 04 sont au vert
- ☐ Une capture du terminal est déposée auprès du formateur

**Si le temps le permet — Exercice 05 : prix TTC**

- `node 05-prix-ttc.js 100 20` affiche `Prix TTC : 120.00 €`.
- Formule : prix HT × (1 + taux / 100).
- Arrondi à deux décimales avec `toFixed(2)`.
- Tests : `node --test 05-prix-ttc.test.js`.

**Pour aller plus loin**

- Ouvrez la page `typeof` de MDN (MDN Web Docs, s. d.-c), puis sa version anglaise (sélecteur de langue en haut de la page).

- Trouvez dans le tableau ce que donne `typeof null`, puis vérifiez-le dans le REPL.

## À retenir — cet après-midi

> À retenir
>
> - **Tests** : `node --test fichier.test.js` teste un exercice.
> - **Test rouge** : `+` ce que vous affichez ; `-` ce qui est attendu.
> - **Littéral de gabarit** : `` `Bonjour ${firstName}` `` insère une valeur.

Le détail, notion par notion :

- **Tests** : `node fichier.js` lance le programme ; `node --test fichier.test.js` lance ses tests ; un test rouge se lit ligne `+` (ce que vous affichez) et ligne `-` (ce qui est attendu).
- **Littéral de gabarit** : il s'écrit entre accents graves ; `${…}` insère une valeur ou un calcul ; avec des guillemets, rien n'est remplacé.

## Débrief (10 min)

- Que montre la ligne `-` d'un test rouge ?
- Quelle différence entre `'…'` et `` `…` `` ?
- Quel exercice vous a demandé le plus d'essais, et pourquoi ?

## Sources

Apple. s. d. « Show or hide filename extensions on Mac ». Consulté le 3 octobre 2026. https://support.apple.com/guide/mac-help/show-or-hide-filename-extensions-on-mac-mchlp2304/mac.

MDN Web Docs. s. d.-a. « console : méthode statique log() ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/API/console/log_static.

MDN Web Docs. s. d.-b. « Égalité stricte (===) - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Operators/Strict_equality.

MDN Web Docs. s. d.-c. « L'opérateur typeof - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Operators/typeof.

MDN Web Docs. s. d.-d. « Littéraux de gabarits - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Template_literals.

MDN Web Docs. s. d.-e. « Types et grammaire - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Guide/Grammar_and_types.

Microsoft. s. d.-a. « Common file name extensions in Windows ». Consulté le 3 octobre 2026. https://support.microsoft.com/en-us/windows/common-file-name-extensions-in-windows-da4a4430-8e76-89c5-59f7-1cdbbc75cb01.

Microsoft. s. d.-b. « Download Visual Studio Code ». Consulté le 26 septembre 2026. https://code.visualstudio.com/download.

Microsoft. s. d.-c. « Installing Visual Studio Code on Linux ». Consulté le 3 octobre 2026. https://code.visualstudio.com/docs/setup/linux.

Microsoft. s. d.-d. « Installing Visual Studio Code on macOS ». Consulté le 3 octobre 2026. https://code.visualstudio.com/docs/setup/mac.

Microsoft. s. d.-e. « Installing Visual Studio Code on Windows ». Consulté le 3 octobre 2026. https://code.visualstudio.com/docs/setup/windows.

Microsoft. s. d.-f. « Set up GitHub Copilot in VS Code (Remove AI features) ». Consulté le 26 septembre 2026. https://code.visualstudio.com/docs/copilot/setup.

Microsoft. s. d.-g. « Workspace Trust ». Consulté le 8 octobre 2026. https://code.visualstudio.com/docs/editing/workspaces/workspace-trust.

nvm-sh. s. d. « nvm : Node Version Manager ». Consulté le 26 septembre 2026. https://github.com/nvm-sh/nvm.

OpenJS Foundation. s. d.-a. « Node.js — Download Node.js ». Consulté le 26 septembre 2026. https://nodejs.org/en/download.

OpenJS Foundation. s. d.-b. « Node.js — Node.js Releases ». Consulté le 26 septembre 2026. https://nodejs.org/en/about/previous-releases.

OpenJS Foundation. s. d.-c. « Process | Node.js Documentation ». Consulté le 26 septembre 2026. https://nodejs.org/api/process.html.

OpenJS Foundation. s. d.-d. « Test runner | Node.js Documentation ». Consulté le 26 septembre 2026. https://nodejs.org/api/test.html.

Schniz. s. d.-a. « fnm : Commands (docs/commands.md) ». Consulté le 26 septembre 2026. https://github.com/Schniz/fnm/blob/master/docs/commands.md.

Schniz. s. d.-b. « fnm : Fast and simple Node.js version manager ». Consulté le 26 septembre 2026. https://github.com/Schniz/fnm.

StackBlitz. s. d.-a. « StackBlitz | Instant Dev Environments ». Consulté le 26 septembre 2026. https://stackblitz.com/.

StackBlitz. s. d.-b. « WebContainers Browser Support ». Consulté le 26 septembre 2026. https://developer.stackblitz.com/platform/webcontainers/browser-support.